

Documentación Técnica

Microservicio: billing-service

Sistema de Gestión de Co-working

Ingeniería de Computación | 7mo Semestre

Integrantes del equipo	Información de entrega
1. Brian Berbey V-30.914.322	Lenguaje / Framework: Node.js 22 / Express 4 Puerto del servicio: 8002 Fecha de entrega: <u>02/06/2026</u> Link del repositorio: <u>https://github.com/lpzzlnrd/Coworking.git</u>
2. Samuel Rojas V-30.855.814	
3. Leonardo Correa V-30.889.380	
4.	
5.	

Rubrica de evaluación

Criterio	Puntaje	Obtenido
README funcional — instrucciones claras para levantar el proyecto	4	
Tabla de endpoints con ejemplos de request y response	4	
Análisis del algoritmo con complejidad $O()$ y ejemplo numérico	6	
Arquitectura y justificación del lenguaje elegido	4	
Evidencia de funcionamiento (capturas de pantalla)	2	
TOTAL	20	

1 Descripción del microservicio

Que deben responder en esta sección

Expliquen con sus propias palabras que hace este microservicio dentro del sistema.

¿Cuál es su responsabilidad específica dentro del co-working?
¿Por qué existe como servicio separado y no dentro de otro microservicio?
¿Con qué otros servicios se comunica y de qué forma?
Máximo 1 página. Sean concisos — un párrafo por pregunta es suficiente.

1.1 Responsabilidad del servicio

Describan en 3-5 oraciones que hace este servicio y que NO hace (sus límites).

billing-service es el microservicio encargado de gestionar la facturación del sistema de co-working. Su responsabilidad principal es crear facturas para miembros, consultar facturas, cambiar estados de pago y generar reportes de ingresos. También permite marcar facturas vencidas cuando su fecha límite ya pasó. No registra usuarios, no autentica credenciales, no emite JWT y no administra reservas; esas responsabilidades pertenecen a otros servicios.

1.2 ¿Por qué existe como microservicio separado?

Justifiquen la separación: ¿qué problema tendríamos si esta lógica viviera en otro servicio?

Existe como microservicio separado porque la lógica de facturación tiene reglas, datos y evolución propios: estados PENDING, PAID y OVERDUE; reportes de ingresos; historial de cobros por miembro; y operaciones masivas sobre facturas vencidas. Si esta lógica viviera dentro del servicio de autenticación o de reservas, esos servicios quedarían acoplados a reglas contables y a consultas SQL que no pertenecen a su dominio. Separarlo facilita probarlo, desplegarlo y escalarlo de forma independiente.

1.3 Integración con otros servicios

Con quien se comunica, como verifica el JWT y que datos intercambia.

El servicio se integra principalmente con la base de datos PostgreSQL compartida, donde persiste la tabla invoices. A nivel de dominio intercambia memberId con el resto del sistema para asociar facturas a usuarios ya autenticados. No verifica JWT directamente: confía en que un API gateway o cliente autorizado envíe el memberId correcto después de autenticar al usuario. También puede ser consumido por el frontend o por otros servicios para consultar facturas y reportes.

2 Tabla de endpoints

Que deben incluir en esta sección

Todos los endpoints que expone su servicio, sin excepción.

Para cada endpoint: método HTTP, URL completa, descripción breve.

Si requiere token (sí/no) y qué rol necesita (cualquiera / solo admin).

Al menos UN ejemplo de request body y response JSON por endpoint principal.
Si tienen mas de 6 endpoints, agreguen filas a la tabla. No omitan ninguno.

2.1 Endpoints del servicio

Metodo	URL	Descripción	Token	Ro l
GET	http://localhost:8002/health	Health check del servicio	No	N/A
POST	http://localhost:8002/invoices	Crea una factura en	No	N/A
GET	http://localhost:8002/invoices	Lista facturas con	No	N/A
GET	http://localhost:8002/invoices/:id	Obtiene una factura por	No	N/A
PATCH	http://localhost:8002/invoices/:id/pay	Marca una factura como	No	N/A
PATCH	http://localhost:8002/invoices/:id/overdue	Marca una factura	No	N/A
POST	http://localhost:8002/invoices/overdue-sweep	Marca masivamente como OVERDUE las facturas vencidas	No	N/A
GET	http://localhost:8002/reports/revenue?period=monthly&from=2026-01-01T00:00:00Z&to=2026-07-01T00:00:00Z	Reporte de ingresos por periodo	No	N/A
GET	http://localhost:8002/reports/members/:memberId/billing	Historial de facturación de un miembro	No	N/A

2.2 Ejemplo completo — request y response del endpoint principal

Elijan el endpoint más importante y muestren el JSON completo.

Método y URL del endpoint elegido

POST http://localhost:8002/invoices

```
// Request body:
```

```
{
  "memberId": "550e8400-e29b-41d4-a716-446655440000",
  "amount": 125.50,
  "description": "Monthly desk rental",
  "dueDate": "2100-01-01T00:00:00.000Z"
}
```

```
// Response (HTTP 201):
{
  "id": "c1c1c1c1-1111-2222-3333-444444444444",
  "memberId": "550e8400-e29b-41d4-a716-446655440000",
  "amount": 125.5,
  "description": "Monthly desk rental",
  "status": "PENDING",
  "dueDate": "2100-01-01T00:00:00.000Z",
  "paidAt": null,
  "createdAt": "2099-12-01T00:00:00.000Z",
  "updatedAt": "2099-12-02T00:00:00.000Z"
}
```

3 Análisis del algoritmo implementado

Que deben responder en esta sección

Esta sección vale 4 puntos — es la más importante del documento.

Expliquen el algoritmo que implementaron en su servicio:

¿Qué hace paso a paso (con palabras, no solo código)?

¿Cuál es su complejidad $O()$ y por qué?

¿Qué pasaría si usaran la alternativa naïve (el for simple)?

Un ejemplo numérico concreto con datos reales del sistema.

No es suficiente pegar el código. Deben EXPLICAR que hace y demostrar que entienden la complejidad.

3.1 Nombre y descripción del algoritmo

Nombre, en que archivo vive y que problema resuelve en el sistema.

Nombre: barrido SQL de facturas vencidas (bulk overdue sweep). Vive en `src/services/invoiceService.js` como `bulkMarkOverdue()` y en `src/repositories/invoiceRepository.js` como `markOverdueInvoices(now)`. Resuelve el problema de detectar todas las facturas pendientes cuya fecha de vencimiento ya pasó y actualizarlas a OVERDUE sin recorrerlas una por una desde `Node.js`.

3.2 Como funciona paso a paso

Explicación en lenguaje natural. Pueden usar diagrama, pseudocódigo o pasos numerados.

1. El endpoint POST /invoices/overdue-sweep llama a invoiceService.bulkMarkOverdue().
2. El servicio obtiene la fecha actual con new Date().
3. El repositorio ejecuta una sola sentencia SQL UPDATE sobre la tabla invoices.
4. La condición filtra status = 'PENDING' y due_date < now.
5. PostgreSQL localiza las filas usando índices sobre status y due_date.
6. Solo las facturas que cumplen ambas condiciones cambian a OVERDUE.
7. La operación devuelve rowCount, es decir, cuántas facturas fueron actualizadas.
8. El servicio responde con { "updatedCount": n }.
9. No se cargan todas las facturas en memoria de la aplicación.

3.3 Comparación de complejidad O()

Llenen la tabla comparativa con su algoritmo y la alternativa sin algoritmo.

Su algoritmo	Alternativa naive (el for simple)
Nombre: UPDATE filtrado con índices en PostgreSQL	Nombre: recorrer todas las facturas en Node.js
Complejidad: $O(\log n + k)$, donde k son las facturas vencidas actualizadas	Complejidad: $O(n)$ para revisar todas, más k actualizaciones
Con 100 elementos hace: búsqueda indexada y actualiza solo las vencidas	Con 100 elementos hace: revisa 100 facturas en la aplicación
Con 10,000 elementos hace: evita cargar las 10,000 filas; delega el filtro a la BD	Con 10,000 elementos hace: recorre 10,000 registros y multiplica roundtrips

3.4 Ejemplo numérico con datos reales del sistema

Usen un ejemplo concreto de su servicio: reservas, espacios, facturas, etc.

Ejemplo: hay 10,000 facturas en invoices. De ellas, 3,200 están PENDING y solo 450 tienen due_date menor que la fecha actual. El algoritmo ejecuta UPDATE invoices SET status = 'OVERDUE' WHERE status = 'PENDING' AND due_date < now. El resultado es updatedCount = 450. La alternativa naive traería o revisaría las 10,000 facturas desde Node.js, comprobaría cada fecha con un if y luego enviaría actualizaciones individuales para las 450 vencidas.

4 Arquitectura y decisiones técnicas

Que deben incluir en esta sección

Diagrama de la arquitectura INTERNA del servicio (capas, flujo de una request).
Justificación del lenguaje o framework elegido para ESTE servicio específicamente.

Como verifica el JWT sin llamar al Auth Service — expliquen el mecanismo.

Una decisión técnica que tomaron y porque (puede ser sobre la BD, el algoritmo o la estructura).

El diagrama puede ser dibujado a mano, capture y pegado aquí.

4.1 Diagrama de arquitectura del servicio

Muestran las capas internas: como fluye una request desde que entra hasta que devuelve respuesta.



Una request entra por Express en src/app.js. Según la URL, se enruta a invoiceRoutes.js, reportRoutes.js o healthRoutes.js. Las rutas validan parámetros y body con validation.js, luego llaman a la capa de servicio. La capa de servicio aplica reglas de negocio como impedir pagar dos veces una factura o impedir marcar como vencida una factura que no esté PENDING. Finalmente, invoiceRepository.js ejecuta SQL contra PostgreSQL y el errorHandler centraliza respuestas 400, 404, 409 y 500.

4.2 Por que eligieron este lenguaje para este servicio

No es suficiente decir que lo conocen. Justifiquen porque es apropiado para la responsabilidad de este servicio.

Node.js con Express es apropiado para este microservicio porque la mayoría del trabajo es I/O: recibir peticiones HTTP, validar JSON, consultar PostgreSQL y devolver respuestas. Express permite definir rutas simples para facturas y reportes, mientras que el driver pg integra directamente con PostgreSQL. Para un servicio de facturación académico, Node también facilita pruebas rápidas con node:test sin depender de un framework de testing externo.

4.3 Como funciona la verificación JWT en este servicio

Paso a paso: como llega el token, como se verifica y que datos se extraen del payload.

billing-service no verifica JWT directamente. Por diseño, no emite ni valida tokens; asume que un API gateway o un cliente previamente autenticado ya validó el usuario y envía el memberId. El servicio solo usa ese memberId como dato de negocio para crear facturas y consultar historial. El tradeoff es que billing-service queda más simple y desacoplado de auth, pero depende de que la capa anterior controle correctamente autenticación y autorización.

4.4 Una decisión técnica importante que tomaron

Algo que decidieron hacer de cierta manera y podría haberlo hecho diferente. Expliquen el tradeoff.

Las migraciones se ejecutan al arrancar el servicio mediante src/db/migrate.js. Esto simplifica el despliegue académico porque el contenedor crea la tabla invoices, índices y trigger updated_at automáticamente. La alternativa sería ejecutar migraciones manuales o desde un pipeline; eso es más controlado en producción, pero agrega pasos y errores posibles durante una demostración.

5 Evidencia de funcionamiento

Que deben incluir en esta seccion

Minimo 3 capturas de pantalla del sistema funcionando.

Captura 1: el servicio levantando con Docker (logs de arranque).

Captura 2: al menos un endpoint funcionando desde Postman con respuesta JSON real.

Captura 3: el health check respondiendo en su puerto (GET /health).

Opcional: captura del endpoint mas complejo o logs del gateway enrutando.

Las capturas deben ser legibles. Si estan muy pequenas, agrandenlas antes de pegar.

Captura 1 — Docker levantando el servicio

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Coworking-Programacion-Matematica> $body = @{  
>>   memberId = "550e8400-e29b-41d4-a716-446655440000"  
>>   amount = 125.50  
>>   description = "Monthly desk rental"  
>>   dueDate = "2100-01-01T00:00:00.000Z"  
>> } | ConvertTo-Json  
>>  
>> Invoke-RestMethod -Method POST `~  
>>   -Uri "http://localhost:8002/invoices" `~  
>>   -ContentType "application/json" `~  
● >>   -Body $body
```

```
id           : 409fa32d-6035-4dbd-a4ee-d54d18fe00ee  
memberId     : 550e8400-e29b-41d4-a716-446655440000  
amount       : 125,5  
description   : Monthly desk rental  
status       : PENDING  
dueDate      : 2100-01-01T00:00:00.000Z  
paidAt       :  
createdAt    : 2026-06-03T02:59:32.048Z  
updatedAt    : 2026-06-03T02:59:32.048Z
```

○ PS C:\Coworking-Programacion-Matematica> █

Captura 3 — Health check del servicio

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Coworking-Programacion-Matematica> Invoke-RestMethod http://localhost:8002/health
status service
-----
ok      billing-service

○ PS C:\Coworking-Programacion-Matematica> |
```

6 Reflexión final del equipo

Que deben responder en esta sección

Esta sección es breve — máximo media página.

¿Qué fue lo más difícil de implementar en este servicio?

¿Que aprendieron del lenguaje o framework que no sabían antes?

¿Si tuvieran más tiempo, que mejorarían o agregarían al servicio?

Sean honestos. No hay respuesta correcta o incorrecta aquí.

Lo más difícil de este servicio fue mantener separadas las reglas de negocio de facturación respecto a la lógica SQL. Por ejemplo, pagar, vencer y listar facturas parecen operaciones simples, pero necesitan validar estados para evitar inconsistencias como pagar dos veces una factura o marcar como vencida una factura ya pagada.

También se aprendió a estructurar un microservicio Node.js por capas: app, rutas, validación, servicios, repositorio, base de datos y middleware de errores. Esa separación hace que los tests puedan probar la lógica con repositorios simulados sin depender siempre de una base real.

Si hubiera más tiempo, sería útil agregar autenticación real en el gateway, pruebas de integración con PostgreSQL, métricas de ingresos más completas y endpoints para anular facturas o registrar métodos de pago. También convendría documentar contratos con OpenAPI/Swagger.

Pruebas ejecutadas: npm test en services/billing-service. Resultado: 18 tests pasaron, 0 fallos.

[illegible]